仮想クリックを用いた ダブルクリックジャッキング検知ツールの考案

金沢 椎凪[1,a)]　成田 匡輝[1]　高田 豊雄[2]

概要：近年, Web ブラウザを標的としたユーザ操作型攻撃の一種として, ダブルクリックジャッキングが報告されている．これは, ダブルクリックの 1 回目のクリック後にページに悪意のある要素を動的に出現させ, 2 回目のクリックでユーザの意図しない操作を実行させる攻撃である．また, この攻撃は従来のクリックジャッキング対策や既存の検出手法では対応が困難であるという問題点を持っている．本研究では, Selenium を用いてクリック操作を仮想的に実行し, クリック前後の UI 変化を動的に解析するダブルクリックジャッキング検知ツールを提案する．提案手法では，1 回目クリック前後の UI 変化を解析してクリックジャッキングの兆候を評価した後，生成された要素に対して 2 回目のクリックを実行し，危険動作の有無を確認することでダブルクリックジャッキングを判定する．評価実験の結果, Accuracy 92.0%, Precision 95.2%, Recall 80.0%, F1-score 87.0%を達成した．

キーワード：ダブルクリックジャッキング, Web セキュリティ, 動的解析

Proposal of Developing a Double-Clickjacking Detection Tool Using Simulated Clicks

SHIINA KANEZAWA[1,a)]　MASAKI NARITA[1]　TOYOO TAKATA[2]

Abstract: Recently, Double-Clickjacking has emerged as a user interaction attack targeting web browsers. In this attack, the user interface is dynamically modified after the first click of a double-click, causing the second click to trigger an unintended operation. Existing clickjacking countermeasures and detection methods are insufficient to address this attack. This paper proposes a Double-Clickjacking detection tool that performs simulated click operations using Selenium and dynamically analyzes UI changes before and after user first clicks. The proposed method analyzes UI changes to identify clickjacking indicators and then executes a second click on the generated element to determine whether malicious actions are triggered. The experimental results achieved 92.0% Accuracy, 95.2% Precision, 80.0% Recall, and an F1-score of 87.0%.

Keywords: Double-Clickjacking, Web Security, Dynamic Analysis

1. はじめに

近年，Web ブラウザを標的としたユーザ操作型攻撃の一種として，ダブルクリックジャッキングが報告されている [1]．ダブルクリックジャッキングは，Web サイトで 1 回目のクリック後にページの UI を動的に変更し，2 回目のクリックによって利用者の意図しない操作を実行させる攻撃である．従来のクリックジャッキングでは，透明な iframe を用いて利用者のクリックを誘導する攻撃が主流であり，X-Frame-Options や Content Security Policy（CSP）などの対策が提案されている [2]．しかし，ダブルクリックジャッキングではクリック後に攻撃要素が生成されるため，これらの対策や既存の検知手法では十分に対応

[1] 岩手県立大学大学院ソフトウェア情報学研究科
Graduate School of Software and Information Science, Iwate Prefectural University
[2] 南山大学理工学部
Faculty of Science and Technology, Nanzan University
[a)] s231x006@s.iwate-pu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

できないという課題がある．

著者らは先行研究において，従来のクリックジャッキング検知手法をダブルクリックジャッキングへ適用し，その有効性を評価した [3]．その結果，従来手法はページの初期状態のみを解析対象としているため，クリック後に生成される悪意ある要素を検知できず，ダブルクリックジャッキングへの適用は困難であることを明らかにした．また，Selenium を用いてクリック操作を仮想的に実行することで，クリック後の UI 変化を取得できることを確認した．Selenium とは, Web アプリケーションや操作を自動化するためのオープンソースツールである [4]．しかし，同研究ではクリック後の UI 変化を利用した検知アルゴリズムの設計には至っておらず，評価対象も 1 種類のサンプルページに限られていたため，提案手法の有効性について, 十分な検証は行われていなかった．

そこで本研究では，Selenium を用いてクリック操作を仮想的に実行し，クリック前後の UI 変化を動的に解析するダブルクリックジャッキング検知ツールを提案する．提案手法では，クリック前後の UI 変化からクリックジャッキングの兆候を評価した後，生成された要素に対して 2 回目のクリックを実行し，リダイレクト，ウィンドウ生成，フォーム送信，ファイルダウンロード，権限要求などの危険動作の有無を確認することでダブルクリックジャッキングを判定する．さらに，多様な攻撃パターンを実装した悪性サンプルサイトおよび正常サンプルサイトを用いて評価実験を行い，提案手法の有効性を検証する．

2. クリックジャッキングの概要

クリックジャッキングは，利用者に意図しない操作を実行させる Web ブラウザを標的とした攻撃である．この攻撃は iframe を利用した従来型の攻撃をはじめ，近年ではダブルクリックジャッキングなどの新たな攻撃手法が報告されている．本節では，従来型クリックジャッキングとダブルクリックジャッキングについて説明する．

2.1 従来型クリックジャッキング

従来型クリックジャッキングは，利用者に本来の意図とは異なる Web ページ上の要素をクリックさせ，意図しない操作を実行させる攻撃である [5]．一般的には，攻撃者が透明または半透明の iframe を正規 Web ページ上に重ね合わせることで，利用者が認識している要素とは異なる要素へクリックを誘導する．その結果，利用者は気付かないまま，情報送信や設定変更などの意図しない操作を実行させられる可能性がある．

従来型クリックジャッキングへの対策として，Web ページの iframe への埋め込みを制限する X-Frame-Options や，Content Security Policy（CSP）の frame-ancestors ディレクティブが提案されている．これらの対策は，攻撃ページによる正規ページの埋め込みを防止することで，iframe を利用した従来型クリックジャッキングに対して有効である．

2.2 ダブルクリックジャッキング

ダブルクリックジャッキングは，2024 年 12 月に Yibelo らによって報告された，クリックジャッキングの派生攻撃である [1]．従来型クリックジャッキングが透明な iframe を利用して利用者のクリックを誘導するのに対し，ダブルクリックジャッキングでは，ダブルクリックの 1 回目のクリック後にページの UI を動的に変更し，2 回目のクリックによって利用者の意図しない操作を実行させる．

ダブルクリックジャッキングでは，1 回目のクリックを契機として，ボタンの差し替えや要素の前面化，iframe の生成などによりクリック対象が動的に変更される．その結果，利用者は意図しない要素をクリックし，リダイレクトや権限要求などの操作を実行させられる可能性がある．

また，本攻撃では攻撃対象がクリック後に初めて生成または変更されるため，iframe の埋め込みを制限する X-Frame-Options や Content Security Policy（CSP）などの従来型クリックジャッキング対策では十分に対応できないという課題がある．

3. 関連研究

従来型クリックジャッキングの検査・検知では，ページの初期状態を対象とした手法が提案されている．Chen らは，HTML および CSS から抽出した特徴量を用いてクリックジャッキングページを判定する手法を提案している [6]．また，Balduzzi らは，クリックジャッキング攻撃を自動的に検査する手法を提案し，攻撃ページを生成して対象ページを解析することで，クリックジャッキング脆弱性を検査した [5]．さらに，高松らは，実際のブラウザ描画を利用した Clickjuggler を提案し，従来手法では検査が困難な Visual クリックジャッキング脆弱性を検査している [7]．しかし，これらの手法はいずれもページの初期状態を対象としており，クリック後の UI 変化は解析対象としていない．

4. 提案手法

本研究では，Selenium を用いてダブルクリック操作を仮想的に実行する．1 回目のクリック後に出現したクリック対象要素について，クリックジャッキングの兆候を解析し，その要素に対する 2 回目のクリック後に実行される危険動作を確認することで，ダブルクリックジャッキングを検知する．図 1 に提案手法の流れを示す．提案手法は，(1) 初期状態の取得，(2)1 回目のクリック後の UI 変化の解析，(3)2 回目のクリック後の危険動作の確認の 3 段階から構成される．

© 2026 Information Processing Society of Japan

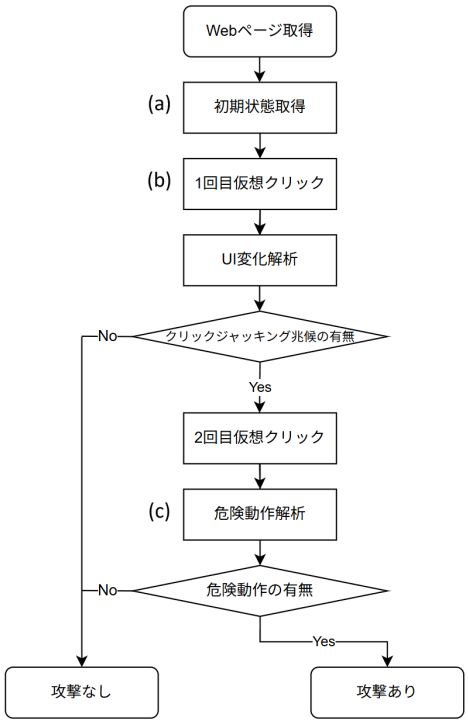


図 1　提案手法の処理フロー

4.1　初期状態の取得 (a)

提案手法では，対象 Web ページを Selenium を用いて読み込み，クリック操作を実行する前の初期状態を取得する．まず，ページ内からクリック可能な要素を抽出し，各要素の情報を取得する．取得する情報は，**id，tag，display，opacity，pointer-events，z-index，座標（x，y），サイズ（width，height），iframe 数，window 数，URL** である．これらの情報を初期状態として保存し，1 回目のクリック後に取得した情報と比較することで，クリックによって発生した UI 変化を解析する．

4.2　1 回目クリック後の UI 変化の解析 (b)

初期状態の取得後，抽出したクリック可能要素に対して 1 回目の仮想クリックを実行する．その後，1 回目のクリックによって新たに出現したクリック対象要素を取得し，当該要素がクリックジャッキングに用いられる可能性のある特徴を持つかを解析する．本研究では，クリックジャッキングの兆候として，**要素の前面化，透明度変化，iframe の出現，pointer-events の変化，popup の出現**の 5 項目を確認する．

従来型クリックジャッキングでは，CSS を利用してクリック対象を隠蔽・重畳する手法や，iframe を利用して別の Web ページを重ねる手法が用いられる [6][8]．また，ダブルクリックジャッキングでは，クリック後にページの状態を動的に変化させることでクリック対象を変更する [1]．そこで本研究では，CSS による表示・操作状態の変化に加え，ページ構造およびブラウザ状態の変化を考慮し，要素の前面化，透明度変化，iframe の出現，pointer-events の変化，popup の出現の 5 項目をクリックジャッキングの兆候として定義する．

要素の前面化は，クリック前後における対象要素の z-index の値を比較することで判定する．透明度変化は，対象要素の opacity の値を比較することで判定する．iframe の出現は，クリック前後の iframe 数を比較し，新たな iframe が生成されたかを確認する．pointer-events の変化は，クリック前後の値を比較し，クリック可能な状態への変化を確認する．popup の出現は，クリック前後の window 数を比較し，新たな window が生成されたかを確認する．

これら 5 項目のうち 1 つ以上のクリックジャッキングの兆候が出現したクリック対象要素において確認された場合，ダブルクリックジャッキングの可能性があるものとして当該要素に対して 2 回目の仮想クリックを実行する．一方，いずれの兆候も確認されなかった場合は，攻撃なしと判定する．

4.3　2 回目クリック後の危険動作の確認 (c)

1 回目のクリック後，クリックジャッキングの兆候が確認された場合，同一のクリック位置に対して 2 回目の仮想クリックを実行する．2 回目のクリック後にページおよびブラウザの状態を取得し，クリック前後の状態を比較することで，危険動作の発生を解析する．本研究では，危険動作として，**リダイレクト，フォーム送信，ファイルダウンロード，window 生成，権限要求**の 5 項目を確認する．

これらを選定した理由として，従来型クリックジャッキングでは，利用者の意図しないクリックによって，機密情報の漏洩や意図しないオンライン取引，ソフトウェアの導入などにつながる可能性が報告されている [7][9]．そこで本研究では，これらの被害につながる動作として，リダイレクト，フォーム送信，ファイルダウンロードの 3 項目を確認する．一方，ダブルクリックジャッキングでは，window を利用してクリック対象を切り替える攻撃や，利用者に意図しない権限操作を実行させる攻撃が報告されている [1]．そのため，window 生成および権限要求を加え，計 5 項目を危険動作として定義する．

リダイレクトは，2 回目のクリック前後における URL を比較し，URL の変化を確認することで判定する．フォーム送信は，クリックによってフォームの送信処理が発生したかを確認する．ファイルダウンロードは，クリックによってファイルのダウンロードが発生したかを確認する．window 生成は，クリック前後の window 数を比較し，新たな window が生成されたかを確認する．権限要求は，クリック後にブラウザの権限要求が発生したかを確認する．

これら 5 項目のうち 1 つ以上の危険動作が確認された場合，ダブルクリックジャッキングによる攻撃ありと判定す

© 2026 Information Processing Society of Japan

る．一方，いずれの危険動作も確認されなかった場合は，攻撃なしと判定する．

5. 提案手法の評価実験

本節では，前節で提案したダブルクリックジャッキング検知手法の有効性を評価する．ダブルクリックジャッキングを模倣した攻撃サンプルサイトおよび正常な動作を行うサンプルサイトに対して提案手法を適用し，攻撃の検知結果から検知性能を評価する．

5.1 実験環境

評価実験は，Google Colaboratory 上に構築した環境で実施した．プログラミング言語には Python 3.12 を使用し，ブラウザの自動操作には Selenium を用いた．Web ブラウザには Chrome for Testing 124，WebDriver には ChromeDriver 124 を使用した．評価対象となるサンプルサイトは Google Colaboratory 上に配置し，HTTP サーバを起動することで Chrome からアクセス可能な環境を構築した．

実験では，Selenium を用いてサンプルサイト内のクリック可能要素を抽出し，クリック操作を仮想的に実行した．各クリックの前後でページおよびブラウザの状態を取得し，前節で定義したクリックジャッキングの兆候および危険動作を解析した．

5.2 評価方法

提案手法の検知性能を評価するため，正常サンプルサイト 50 件と悪性サンプルサイト 25 件の計 75 件からなる評価用データセットを作成した．正常サンプルサイトでは，一般的な Web ページで発生するクリック後の動作を再現した．一方，悪性サンプルサイトでは，1 回目のクリック後にクリックジャッキングの兆候を発生させ，2 回目のクリックによって危険動作が発生するダブルクリックジャッキングを再現した．

5.2.1 悪性サンプルサイトの構成

ダブルクリックジャッキングを再現するため，悪性サンプルサイトを 25 件作成した．悪性サンプルサイトは，1 回目のクリック後に発生する UI 変化を Step1，2 回目のクリック後に発生する危険動作を Step2 とし，Step1 の 5 種類と Step2 の 5 種類を組み合わせて構成した．

Step1 には，4.2 節でクリックジャッキングの兆候として定義した要素の前面化，透明度変化，iframe の出現，pointer-events の変化，popup の出現の 5 種類を用いた．また，Step2 には，4.3 節で危険動作として定義したリダイレクト，フォーム送信，ファイルダウンロード，window 生成，権限要求の 5 種類を用いた．

各悪性サンプルサイトでは，1 回目のクリック後に Step1 のいずれかの UI 変化が発生し，その後，同一座標に対して 2 回目のクリックを行うことで，Step2 のいずれかの危険動作が発生するように構成した．以上の Step1 と Step2 をそれぞれ組み合わせることで，計 25 件の悪性サンプルサイトを作成した．

5.2.2 正常サンプルサイトの構成

正常な Web ページを攻撃サイトとして誤検知しないことを確認するため，正常サンプルサイトを 50 件作成した．正常サンプルサイトは，一般的な UI 変化を再現した 45 件と，悪性サンプルサイトの Step1 と類似する UI 変化を再現した 5 件で構成した．

一般的な UI 変化を再現したサンプルサイトの作成にあたっては，W3C の WAI-ARIA Authoring Practices Guide(APG) で示されている Web UI パターンを参考にした [10]．W3C APG では，Accordion，Alert，Dialog，Menu，Tabs，Tooltip など，Web ページで用いられる代表的な UI パターンが示されている．本研究では，クリック操作に伴って発生する正常な UI 変化を，情報・メッセージ表示，コンテンツ・表示更新，UI 操作・表示切替，検索・確認・結果表示，入力・設定操作の 5 種類に分類した．各分類とサンプル数を表 1 に示す．

表 1 正常サンプルサイトの構成

分類	サンプル数
情報・メッセージ表示	19
コンテンツ・表示更新	13
UI 操作・表示切替	6
検索・確認・結果表示	5
入力・設定操作	2
Step1 類似 UI 変化	5
合計	50

表 1 に示すように，一般的な UI 変化を再現した 45 件に加え，攻撃と類似した UI 変化が正常な Web ページで発生した場合の判定を確認するため，このサンプルサイトを 5 件作成した．

悪性サンプルサイトと類似する UI 変化には，悪性サンプルサイトの Step1 として定義した要素の前面化，透明度変化，iframe の出現，pointer-events の変化，popup の出現の 5 種類を用いた．これらのサンプルサイトでは，1 回目のクリック後にクリックジャッキングの兆候と同様の UI 変化が発生するが，2 回目のクリック後にはリダイレクト，フォーム送信，ファイルダウンロード，window 生成，権限要求のいずれの危険動作も発生しないように構成した．

以上の一般的な UI 変化を再現した 45 件と，Step1 と類似する UI 変化を再現した 5 件の計 50 件を正常サンプルサイトとして評価に用いた．

5.2.3 評価指標

各サンプルサイトに対して提案手法を適用し，悪性サンプルを攻撃ありと判定した場合を True Positive（TP），攻撃

© 2026 Information Processing Society of Japan

なしと判定した場合を False Negative（FN）とする．また，正常サンプルを攻撃なしと判定した場合を True Negative（TN），攻撃ありと判定した場合を False Positive（FP）とする．

提案手法の検知性能を評価するため，Accuracy，Precision，Recall，F1-score の 4 つの評価指標を用いる．これらの評価指標は，TP，TN，FP，FN を用いて算出される [11]．本研究では，Accuracy を式 (1)，Precision を式 (2)，Recall を式 (3)，F1-score を式 (4) により算出する．

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

$$F1\text{-}score = \frac{2 \times Recall \times Precision}{Recall + Precision} \tag{4}$$

5.3 評価結果

正常サンプルサイト 50 件および悪性サンプルサイト 25 件の計 75 件に対して提案手法を適用した．その結果，悪性サンプルサイト 25 件のうち 20 件を攻撃ありと正しく判定し，5 件を攻撃なしと誤判定した．また，正常サンプルサイト 50 件のうち 49 件を攻撃なしと正しく判定し，1 件を攻撃ありと誤判定した．したがって，TP は 20 件，TN は 49 件，FP は 1 件，FN は 5 件となった．提案手法による判定結果を表 2 に示す．

表 2 提案手法の判定結果

TP	TN	FP	FN
20	49	1	5

次に，悪性サンプルサイト 25 件の検知結果を表 3 に示す．表中の○は攻撃を正しく検知したことを，×は攻撃を検知できなかったことを示す．

表 3 悪性サンプルサイトの検知結果

Step1\Step2	リダイレクト	フォーム送信	ダウンロード	window 生成	権限要求
前面化	○	○	○	○	○
透明度変化	○	○	○	○	○
iframe 出現	×	×	×	×	×
pointer-events 変化	○	○	○	○	○
popup 出現	○	○	○	○	○

表 3 に示すように，要素の前面化，透明度変化，pointer-events の変化，popup の出現を Step1 とした 20 件については，Step2 の危険動作の種類によらず，すべて攻撃ありと正しく判定した．一方，iframe の出現を Step1 とした 5 件については，いずれも攻撃を検知できなかった．その結果，悪性サンプルサイト 25 件のうち 20 件が TP，5 件が FN となった．

次に，正常サンプルサイト 50 件の検知結果を表 4 に示す．

表 4 正常サンプルサイトの検知結果

分類	サンプル数	TN	FP
情報・メッセージ表示	19	19	0
コンテンツ・表示更新	13	13	0
UI 操作・表示切替	6	6	0
検索・確認・結果表示	5	5	0
入力・設定操作	2	2	0
Step1 類似 UI 変化	5	4	1
合計	50	49	1

表 4 に示すように，一般的な UI 変化を再現した 45 件については，すべて攻撃なしと正しく判定した．また，Step1 と類似する UI 変化を再現した 5 件については，4 件を攻撃なしと正しく判定した一方，1 件を攻撃ありと誤判定した．この 1 件は，popup の出現を再現した正常サンプルサイトであった．その結果，正常サンプルサイト 50 件のうち 49 件が TN，1 件が FP となった．

最後に，表 2 に示した TP，TN，FP，FN を基に，5.2.3 節で示した式 (1)〜式 (4) を用いて Accuracy，Precision，Recall，F1-score を算出した．各評価指標の算出結果を表 5 に示す．

表 5 提案手法の評価結果

Accuracy	Precision	Recall	F1-score
92.0%	95.2%	80.0%	87.0%

その結果，Accuracy は 92.0%，Precision は 95.2%，Recall は 80.0%，F1-score は 87.0%となった．

6. 考察

評価実験の結果，提案手法の Accuracy は 92.0 %，Precision は 95.2 %，Recall は 80.0 %，F1-score は 87.0 %となった．悪性サンプルサイト 25 件のうち 20 件を攻撃ありと正しく判定したことから，クリック前後の状態変化を解析することで，ダブルクリックジャッキングを検知できることを確認した．一方，悪性サンプルサイト 5 件を攻撃なしと判定する FN と，正常サンプルサイト 1 件を攻撃ありと判定する FP が発生した．

FN となった 5 件は，いずれも iframe を Step1 とした攻撃サンプルであった．本手法では，1 回目のクリック前後における iframe 数の変化から iframe の出現を検知しており，今回の実験においても，5 件すべてで iframe の出現を捉えることができた．しかし，出現した iframe 内で 2 回目のクリック後に発生する危険動作を検知することができず，攻撃なしと判定された．このことから，iframe の出現検知ではなく，iframe 内部で発生する危険動作の解析方法に課題があることが明らかとなった．

© 2026 Information Processing Society of Japan

また，FP となった 1 件は，正常な popup の生成を攻撃の兆候として検知したことによるものであった．本手法では，クリック前後の window 数を比較し，新たな window が生成された場合に popup の出現として検知する．そのため，特に，本実験では攻撃の Step1 と同様に popup を生成する正常サンプルを評価対象に含めたことで，window 数の変化のみを用いた判定では正常動作と攻撃を識別できないことを確認した．このことから，window 数の変化のみでは，正常な popup とダブルクリックジャッキングに利用される popup を十分に区別できないことが分かった．

以上より，提案手法によってダブルクリックジャッキングの検知が可能であることを確認した一方で，iframe を用いた攻撃の見逃しと，正常な popup に対する誤検知が課題として明らかとなった．今後は，iframe の検知方法を改善するとともに，popup については window 数だけでなく，生成された window の位置やサイズ，クリック対象との関係などを考慮することで，検知精度の向上を図る必要がある．

7. おわりに

本研究では，ダブルクリックジャッキングを対象とし，Selenium を用いてクリック操作を仮想的に実行することで，攻撃を検知する手法を提案した．提案手法では，ページの初期状態からクリック可能な要素を抽出し，1 回目の仮想クリックを実行する．その後，1 回目のクリックによって出現したクリック対象要素を調査し，要素の前面化，透明度変化，iframe の出現，pointer-events の変化，popup の出現の 5 項目からクリックジャッキングの兆候を確認する．兆候が確認された場合には，同一のクリック位置に対して 2 回目の仮想クリックを実行し，その後に発生する危険動作を解析することで攻撃の有無を判定する．

提案手法の有効性を評価するため，正常サンプルサイト 50 件と悪性サンプルサイト 25 件の計 75 件を用いて評価実験を行った．その結果，TP は 20 件，TN は 49 件，FP は 1 件，FN は 5 件となり，Accuracy は 92.0%，Precision は 95.2%，Recall は 80.0%，F1-score は 87.0%となった．この結果から，作成した評価用サンプルサイトにおいて，1 回目のクリック後に出現したクリック対象要素を調査し，その後の危険動作を解析することで，ダブルクリックジャッキングを検知可能であることを確認した．

一方，iframe を Step1 とした 5 件では，iframe の出現自体は検知できたものの，iframe 内で 2 回目のクリック後に発生する危険動作を検知できず，FN となった．また，正常な popup の生成を攻撃として判定する FP が 1 件発生した．このことから，iframe 内部で発生する危険動作の解析方法と，正常な popup と攻撃に利用される popup を識別する方法に課題があることが明らかとなった．今後は，iframe 内部の解析方法を改善するとともに，popup の生成位置やサイズなどの情報を考慮した判定方法を検討し，検知精度の向上を図る．

また, 本実験では，提案手法の基本的な検知性能を確認するため，自作した正常サンプルサイト 50 件および悪性サンプルサイト 25 件を評価対象とした．そのため，実際の Web サイトにおける複雑な DOM 構造や JavaScript による非同期処理などを十分に再現できていない可能性がある．今後は，実際の Web サイトや公開されている攻撃 PoC を対象とした評価を行い，実環境における提案手法の有効性を検証する必要がある．

参考文献

[1] Yibelo, P. : “DoubleClickjacking: A New Era of UI Redressing,” URL：https://www.evil.blog/2024/12/doubleclickjacking-what.html （参照 2026-08-16）.

[2] 情報処理推進機構（IPA）： “クリックジャッキング,” URL：https://www.ipa.go.jp/security/vuln/websecurity/clickjacking.html （参照 2026-08-16）.

[3] 金沢椎凪, 成田匡輝, 高田豊雄 ：“ダブルクリックジャッキングに対する検出ツールの考案,” 情報処理学会研究報告， Vol.2025-CSEC-110，No.22，pp.1–6（2025）.

[4] Selenium Project : “Selenium Documentation,” URL：https://www.selenium.dev/documentation/ （参照 2026-08-16）.

[5] Balduzzi, M., Egele, M., Kirda, E., Balzarotti, D. and Kruegel, C. : “A Solution for the Automated Detection of Clickjacking Attacks,” *Proc. of ASIACCS 2010*, pp.135–144（2010）.

[6] Chen, B. and Shi, Y. : “Malicious Hidden Redirect Attack Web Page Detection Based on CSS Features,” *Proc. of 2018 IEEE 4th International Conference on Computer and Communications (ICCC)*, pp.1155–1159（2018）.

[7] Takamatsu, Y. and Kono, K. : “Detection of Visual Clickjacking Vulnerabilities in Incomplete Defenses,” *Journal of Information Processing*, Vol.23, No.4，pp.513–524（2015）.

[8] OWASP Foundation : “Clickjacking,” URL：https://owasp.org/wwwcommunity/attacks/Clickjacking（参照 2026-08-09）.

[9] Narayanan, A. S. : “Clickjacking Vulnerability and Countermeasures,” *International Journal of Applied Information Systems*, Vol.4，No.7，pp.7–10（2012）.

[10] W3C Web Accessibility Initiative : “ARIA Authoring Practices Guide (APG),” World Wide Web Consortium, URL：https://www.w3.org/WAI/ARIA/apg/（参照 2026-08-17）.

[11] Verkerken, M., D’hooge, L., Wauters, T., Volckaert, B. and De Turck, F. : “Towards Model Generalization for Intrusion Detection: Unsupervised Machine Learning Techniques,” *Journal of Network and Systems Management*, Vol.30，Article 12（2022）.

© 2026 Information Processing Society of Japan